

Vulnerability and Trustworthiness in AI-Generated Code

yasir_habib@berkeley.edu

jennifer_nishimura@berkeley.edu

UC Berkeley School of Information

W266 Summer 2026

Abstract

Large Language Models often rely on semantic patterns rather than structural reasoning when detecting and mitigating code vulnerabilities, leading to poor code generation and higher hallucination rates. This paper evaluates CodeBERT for vulnerability classification and hallucination detection in LLM-generated remediations using custom datasets validated by an AST-based mining pipeline and a gold-standard dataset. Raw code sequences performed poorly for vulnerability detection ($F1=0.5945$ and $AUROC=0.6237$) while single-CWE fine tuning ($F1=0.957$) and an Abstract Syntax Tree representation ($F1=0.9842$) achieved strong performance, but generalized poorly to unseen CWEs. For hallucination detection, fine-tuning on a small dataset collapsed to a near-constant classifier, while frozen CodeBERT embeddings combined with TF-IDF improved the Mathew's Correlation Coefficient [7] from 0.426 to 0.530. Further scaling the dataset stabilized fine-tuning but did not outperform the CodeBERT+TF-IDF approach. Overall, Coderbert is an effective choice for vulnerability and hallucination tasks within targeted CWE subsets but remains limited in cross-CWE generalization.

1 Introduction

LLMs have exponentially grown in their importance in software development and we will soon generate more code than humans can thoroughly review and fact check. Because of this, it is vital that we evaluate models on tasks related to code vulnerabilities and this paper seeks to evaluate two distinct but related security tasks: identifying whether a snippet of code contains a vulnerability, and generating remediations once a vulnerability is found. Both tasks rely on the idea that the model will reason why the code is unsafe, its data flow, and structural logic based instead of simple pattern matching. This paper focuses on and investigates whether CodeBERT, a bimodal encoder pretrained for the generation of code can classify against two related problems drawn from the C/C++ Juliet Test Suite and measure its ability to 1) identify vulnerabilities and 2) identify hallucinations when fixing vulnerabilities. We treat these two tasks as a part of the same question of whether a pretrained code model can capture structural understanding to identify issues and reasonably mitigate them using code.

2 Related Work

Huang et al. (2026) revealed that models rely on surface-level syntactic patterns rather than reasoning. Additionally, while models achieve high accuracy when distinguishing code, the performance degrades when classifying vulnerable functions against their patched counterparts demonstrating that SFT is insufficient to overcome these limitations.

Li et al. (2026) evaluated the effectiveness of LLM-based vulnerability detection at the project scale. Achieving low recall and false discovery rates exceeding 85%, they concluded that shallow interprocedural reasoning and misidentified source/sink pairs limit scalability.

Ribeiro et al. (2025) found that an LLM's internal states and attention weights contain hidden signals correlating with code correctness. This indicates the possibility of building a classification head to detect hallucinated code without requiring model retraining.

Li et al. (2025) analyze the out-of-distribution (OOD) capabilities of LLMs trained on vulnerability datasets when detecting the Common Weakness Enumeration (CWE). They show

that datasets contain substantial noise, duplication, and correlations, which can inflate in-distribution performance. Using the BenchVul benchmark, they found that these gains collapse out-of-distribution, motivating the need for independent benchmarks. These studies highlight that datasets overfit to specific patterns, rely on superficial syntactic cues, and generate hallucinated security explanations. Building on these findings, our work evaluates the effectiveness of using CodeBERT for vulnerability and hallucination detection.

3 Dataset

This study uses custom datasets and benchmarks to assess both tasks. The dataset is derived from the C/C++ Juliet Test Suite, a benchmark for evaluating vulnerability detection and remediation.

3.1 Core Dataset

The dataset was constructed by prompting Qwen2.5-Coder-1.5B-Instruct to remediate vulnerable C/C++ functions from the Juliet Test Suite, producing 5868 samples. The generated code was extracted from the markdown, parsed into an AST, and validated against an allowlist of standard APIs to identify API usage and syntax errors. To establish reliable ground truth, 64 samples spanning hallucination categories were manually reviewed to create a “gold dataset”, which served as the primary benchmark for evaluating downstream hallucination classifiers.

3.2 Supplement Dataset

An additional dataset was generated leveraging a Qwen decoder model on the CodeXGLUE defect detection benchmark. It follows a similar structure as the primary dataset, recording generated code, compilation results, and hallucination labels to evaluate behavior.

4 Methodology

CodeBert was selected as the model of choice because it is bimodal pre-trained on both natural language and programming language. Its selection was motivated by the prior literature, its ability to capture semantic and syntactic code representations, and its strong performance on downstream code classification tasks [5].

4.1 Vulnerability

4.1.1 Vulnerability Baseline

The baseline establishes a benchmark by fine-tuning CodeBERT for vulnerability detection. This experiment follows the paradigms adopted in modern LLM-based vulnerability research [2]. The dataset with 2,131 benign and 1,869 vulnerable snippets, were preprocessed in order to remove duplicate functions: stripped C-style comments, whitespace, and truncated input sequences to a maximum length of 512 tokens. To analyze the internal code representations, 768-dimensional hidden states of the final-layer were extracted. t-Distributed Stochastic Neighbor Embedding (t-SNE) is applied to project the embedding into a 2D space to visualize whether the model learned representations of vulnerable and benign code samples.

4.1.2 Single CWE

To evaluate performance for a specific vulnerability type, this experiment focused on CWE-119. Duplicate code snippets were purged by computing SHA-256 digests in order to minimize the semantic trap and dataset noise [1]. This step is performed to improve data consistency and reduce label noise. The sanitized dataset was partitioned into an 80/10/10 train, validation, and

test split A NN classifier was built in order to extract 768-dimensional representations and project through 3,072-dimensional dense layers with dropout. To address the class imbalance, class weights were calculated from baseline testing (1.90 minority and 0.68 majority) and applied to the Cross-Entropy Loss function.

4.1.3 Abstract Syntax Tree

An Abstract Syntax Tree approach is used to improve the models understanding of the structure and logic [6]. Tree-sitter parsing library with C++ grammar was used to generate ASTs for each function. A depth-first traversal was applied to extract structural node types (function_definition and if_statement). To prevent the model from relying on semantic cues [1], identifiers and literals were replaced with generic tokens. However, keywords such as malloc, free, and strcpy were preserved. Additionally, Juliet-specific macros and indicators were removed before tokenization to help the model focus on patterns. To address the challenge of detecting OOD vulnerability types [4], a zero-shot cross-CWE split was used to evaluate the robustness of the approach with a training set containing 7 categories and the test set containing 42 unseen categories.

4.2 Hallucination

Hallucination detection was performed on the generated dataset using a baseline model of TF-IDF, fine-tuned CodeBERT and CodeBERT + TF-IDF. The dataset was run in two sizes, a smaller sample set of ~1k rows and a larger ~5k row dataset. Out of 5.4k samples, almost 500 were removed as empty extractions or duplicates resulting in a balanced dataset. The data was split on a group aware partition into 49 CWEs for training and 9 CWEs for testing.

4.2.1 Hallucination Baseline Model

To establish a baseline for hallucination detection, a linear classification pipeline was implemented using term frequency-inverse document frequency (TF-IDF) features paired with a regularized logistic regression classifier.

4.2.2 CodeBERT Fine-Tuned with LoRA

CodeBERT was fine-tuned using Low-Rank Adaptation (LoRA) on its query and value projection layers, along with a trainable classification head. We used a class-weighted loss to account for label imbalance and evaluated models using MCC rather than F1, since a trivial classifier that always predicted "hallucinated" scored an artificially high F1.

4.2.3 CodeBERT (Frozen-Weights) + TF-IDF

To test whether CodeBERT struggled with unstructured code, we evaluated the model as a frozen feature extractor and used CodeBERT in conjunction with TF-IDF to form a combined representation with a logistic regression classifier. This model was used to determine whether CodeBERT can provide value above the TF-IDF baseline by concatenating the CodeBERT [CLS] embedding with the baseline TF-IDF features under an otherwise identical logistic regression classifier.

5 Results

5.1 Vulnerability

Method	Validation Loss	Precision	Recall	F1 Score	ROC
Baseline	0.7217	0.5308	0.6757	0.5945	0.6237
Single CWE	0.0596	0.9775	0.9329	0.9547	————
AST	0.1787	0.9811	0.9873	0.9842	0.9989
AST OOD	0.6641	0.4634	0.8830	0.6078	0.6164

Table 1: Vulnerability Detection Performance

5.1.1 Baseline

Fine-tuning baseline sequence classifiers on raw source code yielded limited performance, with the best model achieving an F1 score of 0.5945 and an AUROC of 0.6237 at Epoch 3. This suggests the model relied on surface-level token patterns rather than program structure, resulting in moderate recall (0.6757) but low precision (0.5308) and a high false-positive rate.

As shown in Figure 1, the model begins to overfit after Epoch 3, with training loss decreasing ($0.5752 \rightarrow 0.4940$) while validation loss increases ($0.7217 \rightarrow 0.7634$). The t-SNE visualization shows substantial overlap between vulnerable and benign embeddings, indicating weak class separability, while the Precision–Recall curve reveals that the precision declines rapidly as recall increases. These results align with Huang (2026) and show that raw sequence classification fails to learn robust representations for vulnerability detection [1].

5.1.2 CWE Specific Detection

The single-CWE experiment demonstrated the effectiveness of targeted fine-tuning for vulnerability detection. Before training, the model exhibited severe majority-class collapse, predicting almost all samples as safe ($F1 = 0.0000$, $FNR = 1.0000$). After preprocessing the dataset by identifying identical code samples through SHA-256 hashing, removing samples with inconsistent labels, and applying class-weighted fine-tuning, the model achieved: accuracy = 96.48%, $F1 = 95.47\%$. This revealed a substantial improvement over the baseline.

As shown in the figure 2 training loss curve, the model converged rapidly during the first epochs, with loss decreasing from 0.1785 in Epoch 1 to 0.0454 by Epoch 4. The performance peaked around Epoch 3, reaching an F1 score of 0.9575 and recall of 95.44%, and following epochs showed diminishing improvements, indicating convergence. The distribution also shifted towards a balanced separation between vulnerable and benign samples. However, these results represent performance on a single vulnerability category, and further evaluation across multiple CWE classes is required to assess complete generalization, aligning with Li, et al results [2].

5.1.3 Abstract Syntax Tree

5.1.3.1 In-Distribution AST Performance

The AST model achieved strong performance when evaluated on CWE categories included during training. By converting source code into structured AST token sequences and abstracting literals into generic tokens, the model learned structural code patterns rather than memorizing surface level failures. Based on evaluation results, the model quickly converged, achieving an F1 score of 0.9842 and AUROC of 0.9992 by Epoch 5. The confusion matrix confirmed strong

classification, with 242 true negatives, 142 true positives, 16 false negatives, and 0 false positives, demonstrating that AST abstraction effectively reduced false vulnerability predictions. These results suggest that structural representations help reduce shortcut learning from identifiers and dataset-specific artifacts [6].

5.1.3.2 Out-of-Distribution (OOD) Generalization

The AST model was further evaluated on unseen vulnerability categories by training on 7 CWEs and testing on 41 unseen CWEs. As shown in Figure 3, performance significantly dropped compared to the in-distribution evaluation, with the best F1 score reaching 0.6079 and AUROC reaching 0.6164. While the recall remained high (0.8830), precision decreased to 0.4635, resulting in higher false positives based on unfamiliar vulnerability patterns. The ROC curve has moderate separation above random classification, while the Precision-Recall curve shows a range of approximately 0.45–0.50 at higher recall levels. These results show that AST representations improve structural learning but are not sufficient for detecting unseen CWE categories without broader cross-CWE training [4].

5.2 Hallucination

For hallucination evaluation metric we used the Matthews Correlation Coefficient (MCC) because it provides a stricter penalty for accidental random guessing or model collapse than standard metrics like accuracy or F1-score (Yao et al).

Method	Evaluation Accuracy	Evaluation F1	Evaluation MCC	Gold MCC
TF-IDF/LogReg (Baseline)	0.648	0.568	0.379	0.558
CodeBERT (LoRA)	0.745	0.603	0.472	0.453
CodeBERT (frozen + TF-IDF)	0.744	0.643	0.463	0.439

Table 2: Hallucination Detection Performance (5k rows)

5.2.1 Baseline Model

The baseline pipeline achieved an accuracy of 0.762 and an F1 score of 0.747 on the training split but evaluation performance on the unseen CWE families dropped to an accuracy of 0.648 and a macro F1 score of 0.590. Increasing the dataset size to around 5k rows increased the MCC to 0.379 for the evaluation set and .553 for the gold set and an evaluation of the strongest predictors showed that the marking of a hallucination was the result of structural artifacts like Rust keywords and Java syntax instead of language drift. Conversely, the strongest predictors of "not hallucinated" were Juliet template-structure artifacts rather than indicators of correct security reasoning. To mitigate this, future models should strip structural templates to force pure semantic reasoning. These results reveal that surface-level lexical tokens struggle to generalize across entirely unseen vulnerability families, establishing a clear baseline threshold for future transformer-based detection methods.

5.2.2 CodeBERT Fine-Tuned with LoRA

Fine-tuning CodeBERT with LoRA on a smaller dataset of 1k rows resulted in model collapse with training loss remaining near 0.693 across all epochs which is common of Roberta family models on small datasets. Increasing the dataset size to 5k enabled the model to no longer default

to a constant prediction and resulted in a MCC of 0.472 and a gold set MCC of 0.453. This result beat the TF-IDF model at scale but resulted in model collapse for the sample set.

5.2.3 CodeBERT (Frozen Weights) + TF-IDF

On the smaller 1k dataset eval-split accuracy rose from 0.648 to 0.676 and MCC from 0.288 to 0.349, while gold-set accuracy rose from 0.703 to 0.766 and MCC from 0.426 to 0.530, a 24% relative improvement on the TF-IDF baseline model. Repeating this on a larger 5k dataset did not reproduce this advantage as the MCC rose slightly to 0.463 but the gold-set MCC declined to 0.439 to below the TF-IDF baseline result on the larger dataset. This is due to the addition of more CWEs and vulnerability types creating noise making it difficult for CodeBERT to pick up signals across many CWE types.

These results demonstrate that across different methods and scales of data, using semantic clues outperformed the use of CodeBERT to look for reasoning within the code. The CodeBERT model struggles to learn on a smaller dataset but increasing the size of the dataset also introduces more CWEs which increases noise and makes it difficult for the model to generalize to many different types of vulnerabilities.

5.3 CodeBERT Performance Evaluation: Vulnerability vs Hallucination

In summary, CodeBERT failed to generalize for the 2 tasks, but for different reasons. Both baselines' performance reflected random guessing. The fine-tuning approach for vulnerability detection and LoRA fine-tuning for hallucination detection collapsed. These results are influenced by the noise and unstructured nature of the datasets in addition to vulnerability mitigations generally resulting in subtle changes within the code ultimately confusing the model. Both models substantially improved performance over the baseline implementing more targeted approaches and methods. Specially for vulnerability detection, data normalization and narrowing the scope to a single CWE category boosted the F1 to 0.9547. Hallucinations performance training on a smaller CWE set and implementing CodeBERT with frozen weights + TF-IDF improved over the baseline.

Both tasks resulted in a similar generalization gap to unseen CWE categories. The AST vulnerability model performed strongly on the in-distribution test set with $F1 = 0.9842$, yet suffered with the unseen out-of-distribution validation test with $F1 = 0.6079$. The hallucination model with its held-out group resulted in $F1$ of 0.596, suggesting the CodeBERT representations hit a comparable generalization ceiling on both tasks, despite model usage.

6. Conclusion

This paper successfully evaluated the performance of CodeBERT on 2 tasks: vulnerability and hallucination detection in LLM-generated remediation. Naive fine-tuning approaches for both tasks resulted in weak model performance. Vulnerability detection improved by narrowing the scope to a single CWE and deduplicating samples while the hallucination detection only improved by bypassing fine-tuning and using a CodeBERT with TF-IDF. Both approaches converged to similar generalization results on unseen CWE categories suggesting a limitation on CodeBERT to generalize to different CWE tasks. Analyzing findings based on the literature review and final metrics represent CodeBERT's ability to identify vulnerabilities but requires adaptation to individual tasks rather than fine-tuning.

References

1. Huang, F., Sun, Y., Zhang, F., Yang, Z., Liu H., Liu Y., (2026). Do Fine-Tuned LLMs Understand Vulnerabilities? An Investigation into the Semantic Trap, <https://doi.org/10.48550/arXiv.2601.22655>
2. Li, F., Jiang, J., Chen, D., Xiong, Y., (2026). LLM-based Vulnerability Detection at Project Scale: An Empirical Study, <https://doi.org/10.48550/arXiv.2601.19239>
3. Ribeiro, F., Spiess, C., Devanbu, P., Nadi, S., (2025). On LLMs' Internal Representation of Code Correctness, <https://doi.org/10.48550/arXiv.2512.07404>
4. Li, Y., et al. (2025). Out of Distribution, Out of Luck: How Well Can LLMs Trained on Vulnerability Datasets Detect Top 25 CWE Weaknesses?, <https://doi.org/10.48550/arXiv.2507.21817>
5. Feng, Z., et al. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages, <https://doi.org/10.48550/arXiv.2002.08155>
6. Zhu, B., Tan, H., (2023). VuLASTE: Long Sequence Model with Abstract Syntax Tree Embedding for vulnerability Detection, <https://doi.org/10.48550/arXiv.2302.02345>
7. Yao, J., & Shepperd, M. (2020). *Assessing software defect prediction performance: Why using the Matthews correlation coefficient matters*. In Proceedings of the 24th International Conference on Evaluation and Assessment in Software Engineering (EASE '20), pp. 120–129. <https://doi.org/10.1145/3383219.3383232>

Contributions

Yasir: I worked on the vulnerability aspect of the project. Conducted optuna study in order to test baselines with various models and select CodeBert for the experiment based on best performance and literature review. Worked on the various methods in order to evaluate for vulnerability detection (AST, CWE in and out of order). I also worked on prompt engineering and generating a vulnerability dataset using Qwen decoder model.

Jennifer: I worked on the generation of the hallucination dataset and evaluating CodeBERT for the created data. For dataset creation, I explored multiple methods of tagging the data including an AST parser and the w64devkit. For model evaluation, I ran a baseline model of TF-IDF and two different CodeBERT models, one with LoRA and another with frozen weights + TF-IDF.

Appendix: Figures and Tables

1. Vulnerability Figures:

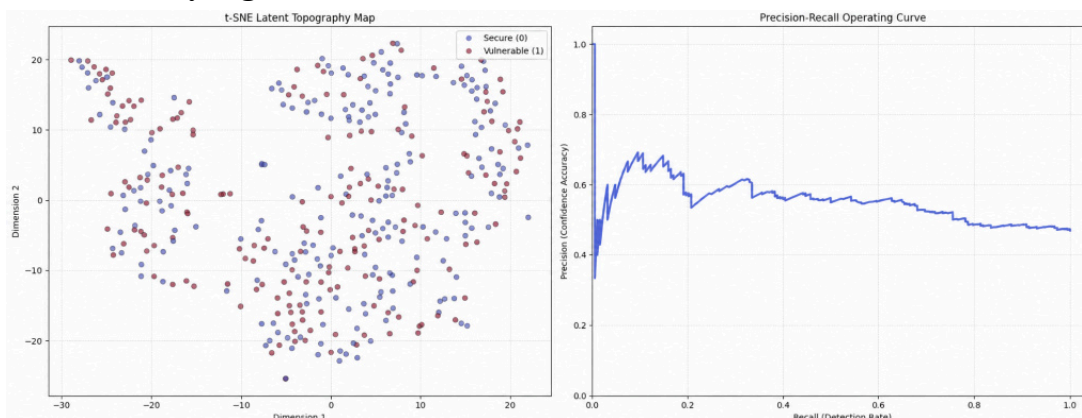


Figure 1: Baseline t-SNE Latent Map & ROC Curve

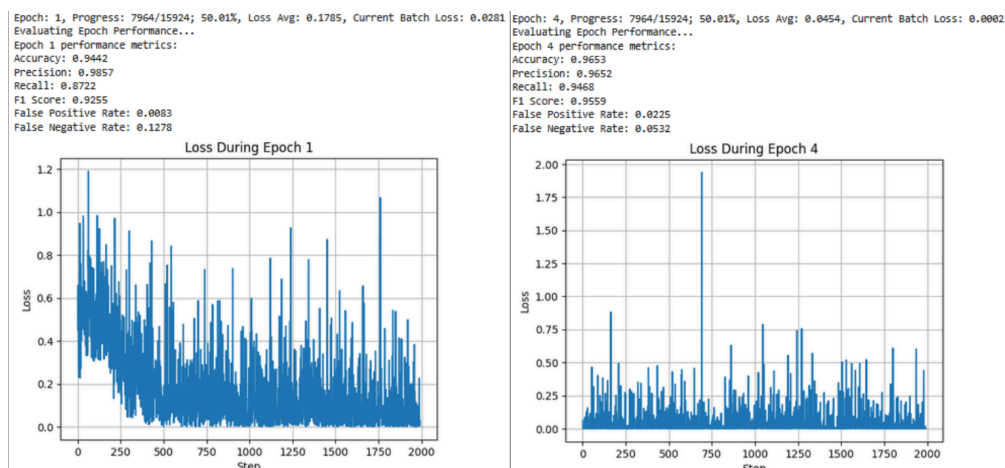


Figure 2: CWE-119 Training Loss Curve

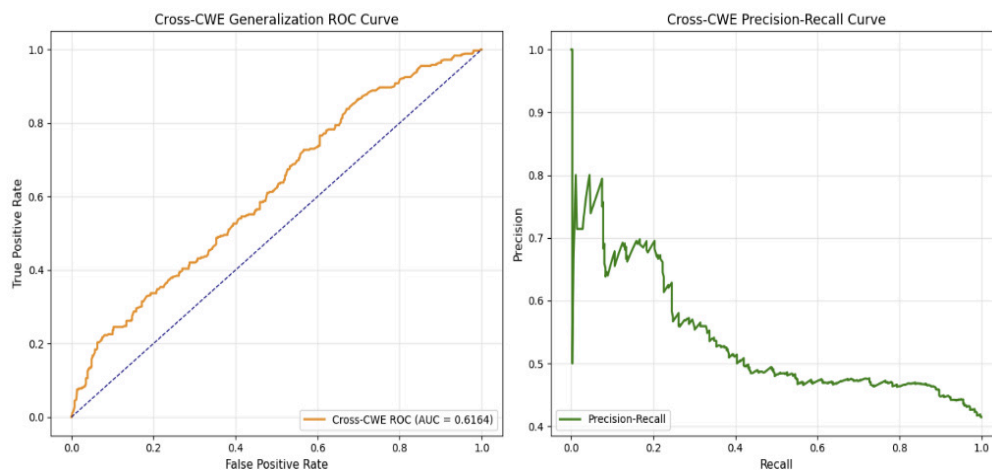


Figure 3: Abstract Syntax Tree Training Loss Curve

2. Hallucinations 1k vs 5k Dataset:

Model	Size	Train Acc	Train F1	Eval Acc	Eval F1	Eval MCC	Gold Acc	Gold F1	Gold MCC
TF-IDF + LogReg <i>(Baseline)</i>	1k 5.4k	0.762 0.736	0.747 0.727	0.648 0.706	0.590 0.568	0.288 0.379	0.703 0.719	0.753 0.743	0.426 0.558
CodeBERT <i>(Fine-tuned LoRA)</i>	1k 5.4k	— —	— —	0.702 0.745	0.030 0.603	0.090 0.472	— 0.656	— 0.676	<i>Collapse</i> 0.453
CodeBERT (Frozen) + TF-IDF	1k 5.4k	0.837 0.785	0.831 0.779	0.676 0.744	0.596 0.643	0.349 0.463	0.766 0.672	0.810 0.704	0.530 0.439

Table 3: Complete 5k vs 1k Hallucination Comparison